Test unitar și test de integrare

Un **test unitar** verifică o singură clasă, fără Spring: o creezi cu `new` și îi dai dependențele de mână. Un **test de integrare** pornește contextul Spring și verifică bean-urile legate între ele. Cel unitar rulează în milisecunde; cel de integrare pornește toată aplicația.

```
@BeforeEach
void setUp() {
    service = new ComandaServiceImpl(comandaRepository, produsService,
            calculatorPret, comandaProvider);
}
```

Asta merge fiindcă `ComandaServiceImpl` își primește dependențele prin constructor, în câmpuri `final`. Cu `@Autowired` pe câmp, `new` ar lăsa câmpurile `null`: doar Spring le-ar putea completa, iar testul ar pica cu `NullPointerException`. Injecția prin constructor face clasa testabilă fără Spring.

Ce e un mock

Un mock e un obiect fals de tipul dependenței. Nu are logică: întoarce valoarea implicită (`null`, `0`, `false`, `Optional.empty()`) până îi spui altceva și ține minte fiecare apel primit.

- **Ce întoarce:** `when(apel).thenReturn(valoare)` fixează răspunsul înainte de test.
- **Ce ține minte:** `verify(mock).metoda(...)` cere ca apelul să fi avut loc o dată; `verify(mock, never())` cere să nu fi avut loc deloc.
- **Cu ce argument:** `ArgumentCaptor` prinde obiectul primit de mock, ca să-l verifici după.

```
ArgumentCaptor<Comanda> captor = ArgumentCaptor.forClass(Comanda.class);

service.finalizeaza(comanda);

verify(comandaRepository).addComanda(captor.capture());
assertEquals(7, captor.getValue().getId());
```

Testul nu citește lista din repository (mock-ul nu are listă), ci verifică ce i s-a trimis.

Forma unui test: Arrange, Act, Assert

Fiecare test are trei părți: pregătești datele și răspunsurile mock-urilor, faci un singur apel, verifici rezultatul.

```
@Test
void adaugaProdusInexistentNuModificaComanda() {
    // Arrange
    Comanda comanda = new Comanda(1);
    when(produsService.findById(99)).thenReturn(Optional.empty());

    // Act
    boolean adaugat = service.adaugaProdus(comanda, 99);

    // Assert
    assertFalse(adaugat);
    assertTrue(comanda.getProduse().isEmpty());
    verify(comandaRepository, never()).addComanda(any());
}
```

La sume, `assertEquals(37.93, total, 0.005)` are al treilea argument, toleranța: un `double` nu iese niciodată exact.

@SpringBootTest și @MockitoBean

`@SpringBootTest` pornește aplicația ca `SpringApplication.run`. `@MockitoBean` pe un câmp înlocuiește în context bean-ul de acel tip cu un mock; restul bean-urilor rămân reale.

```
@SpringBootTest
class CafeneaContextTest {

    @MockitoBean
    private ViewCafenea viewCafenea;

    @MockitoBean
    private ProdusRepository produsRepository;
```

Capcana: `CommandLineRunner`-ul rulează și în `@SpringBootTest`, deci `run()` apelează `viewCafenea.start()`. Pe bean-ul real, meniul așteaptă la `Optiune:` o linie din `Scanner` care nu vine, iar `mvn test` stă blocat. Testul generat de start.spring.io pățește la fel. Cu mock-ul, `start()` nu face nimic și testul trece mai departe.

`ProdusRepository` înlocuit întoarce, după `when`, `Espresso 8.00`, iar `ComandaService` real calculează cu `CalculatorPret` real: `8.00 × 1.21 = 9.68`.

@Order are efect doar când sortează Spring

Spring sortează după `@Order` lista `List<Reducere>` pe care o injectează. Într-un test cu `new` și `List.of(...)`, nu sortează nimeni: ordinea e cea scrisă. Ca testul să verifice `@Order`, sortezi lista cu același comparator ca Spring:

```
List<Reducere> reduceri = new ArrayList<>(
        List.of(new ReducereFidelitate(), new ReducereMeniu()));
AnnotationAwareOrderComparator.sort(reduceri);
```

- Sortată: meniul întâi, `35.00 − 2 = 33.00`, apoi `× 0.95 = 31.35`, cu TVA `37.93`.
- Nesortată: fidelitatea întâi, `35.00 × 0.95 = 33.25`, apoi `− 2 = 31.25`, cu TVA `37.8125`.

`new ArrayList<>(...)` e necesar: `List.of` nu se poate sorta.